

Actividad Obligatoria No. 1

Vibecoding y concurrencia: Análisis de un sistema de delivery simulado con Python threading

Máximo Augusto Bosch, Franco Nicolás Vallejos, Rocío Lorena Chauque, Micaela Valeria Pucca, Nahuel Ezequiel López Ferme

Universidad Nacional de La Matanza Buenos Aires, Argentina

{ rchauque@alumno.unlam.edu.ar }

{ nlopezferme@alumno.unlam.edu.ar }

{ mbosch@alumno.unlam.edu.ar }

{ franvallejos@alumno.unlam.edu.ar }

{ mipuca@alumno.unlam.edu.ar }

Resumen

Se analiza la generación automática de código concurrente en Python 3.10+ mediante las IAs: Gemini 3.1 Pro, Claude Sonnet 4.6, GPT-5 mini y Deepseek v4 Flash. A partir de un prompt técnico detallado, se obtuvieron simulaciones estilo delivery con un flujo productor-consumidor de dos etapas y 12 threads simultáneos. Los códigos fueron evaluados estimando métricas estándar: complejidad ciclomática promedio baja, índice de mantenibilidad moderado-alto, uso de CPU insignificante (por bloqueo en I/O simulado) y consumo de memoria mínimo. No se detectaron deadlocks ni race conditions, aunque existe un riesgo leve de starvation inherente al uso de colas sin prioridad.

1. Introducción

Este análisis replica la metodología de evaluación de herramientas de generación automática de código en el ámbito de la programación concurrente. Se evalúa la aptitud de los modelos Gemini 3.1 Pro, Claude Sonnet 4.6, GPT-5 mini y Deepseek v4 Flash para resolver problemas que exigen el manejo preciso de recursos compartidos, sincronización y exclusión mutua.

El patrón productor-consumidor, formalizado por Dijkstra, requiere mecanismos de sincronización aplicados rigurosamente para evitar estados inconsistentes. El objetivo de este informe es evaluar cualitativa y cuantitativamente los códigos generados,

para ver si cumplen con los requisitos de corrección, eficiencia y mantenibilidad, aplicando los fundamentos de McCabe y Oman & Hagemeister.

2. Desarrollo

2.1. Herramientas de vibecoding seleccionadas

Para esta evaluación se seleccionaron las herramientas de *vibecoding*: **Gemini 3.1 Pro**, un modelo orientado a baja latencia y respuesta rápida, útil para idear soluciones, generar esqueletos de código y recibir sugerencias contextuales “al vuelo” (especialmente práctico en iteraciones cortas y prototipado); **Sonnet 4.5**, enfocado en el flujo de desarrollo dentro del IDE con asistencia en línea (autocompletado, refactorización guiada, explicación de fragmentos, generación de pruebas y comandos tipo “copilot”); **GPT-5 mini**, que es la versión compacta de GPT-5, destacando por su ventana de contexto de 400.000 tokens y su capacidad de razonamiento rápido y eficiente para cargas de trabajo de alto volumen, tareas repetitivas y casos de uso donde el presupuesto es clave; y **Deepseek v4 Flash**, que es un modelo optimizado para alta eficiencia, velocidad y bajo costo, que cuenta con 284.000 millones de parámetros y admite una ventana de contexto de 1 millón de tokens.

2.2. Prompt técnico utilizado

De los diferentes tipos de prompts que se conocen en la literatura, se ha elegido uno basado en el tipo Prompt Patterns, subtipo Input Semantics. La característica distintiva es que permite observar una estructura lógica, clara y coherente con la Ingeniería de Requerimientos utilizada.

La estructura que sigue este prompt técnico es:

- LENGUAJE DE PROGRAMACIÓN
- PLATAFORMA
- ACTORES
- REQUISITOS ESPECÍFICOS
- RESTRICCIONES

A continuación, se puede observar el prompt técnico final:

LENGUAJE DE PROGRAMACIÓN:

- Utilizar Python 3.10+ como lenguaje principal.
- Se permite el uso de la biblioteca estándar únicamente: threading, multiprocessing, queue, asyncio, time, random, logging, collections.
- No se deben usar frameworks externos ni librerías de terceros.

PLATAFORMA:

- Aplicación de consola ejecutable en cualquier sistema operativo con Python 3.10+.
- Sin interfaz gráfica; toda la salida debe realizarse por stdout mediante el módulo logging.

FUNCIONALIDAD A IMPLEMENTAR:

- Desarrollar una simulación de un sistema de delivery estilo Uber Eats / PedidosYa con múltiples actores concurrentes.
- El sistema debe modelar el ciclo completo de un pedido: generación → preparación → entrega → confirmación.
- Debe soportar múltiples instancias simultáneas de cada actor (clientes, cocineros, repartidores).

ACTORES DEL SISTEMA:

- Clientes (Producers): generan pedidos de forma aleatoria con un intervalo configurable. Cada pedido tiene: ID único, nombre del plato, tiempo estimado de preparación y timestamp de creación.
- Cocineros (Workers): toman pedidos de la cola de pedidos pendientes, los "preparan" (simulado con `time.sleep()` proporcional al tiempo estimado) y los colocan en la cola de pedidos listos.
- Repartidores (Deliverers): toman pedidos de la cola de pedidos listos y los "entregan" (simulado con `time.sleep()` aleatorio dentro de un rango configurable). Al completar, registran el tiempo total del pedido.
- Monitor del sistema: hilo dedicado que cada N segundos imprime el estado del sistema: pedidos en cola, pedidos en preparación, pedidos en entrega, pedidos completados, threads activos.

REQUISITOS ESPECÍFICOS:

- La cantidad de clientes, cocineros y repartidores debe ser configurable mediante constantes al inicio del programa.
- Implementar dos colas thread-safe usando `queue.Queue`:
 - `cola_pedidos_pendientes`: entre clientes y cocineros (productor-consumidor nivel 1).
 - `cola_pedidos_listos`: entre cocineros y repartidores (productor-consumidor nivel 2).
- Cada cola debe tener una capacidad máxima configurable. Si la cola está llena, el productor debe bloquearse (comportamiento natural de `queue.Queue` con `maxsize`).
- Implementar un `threading.Event` de apagado (`shutdown_event`) que al activarse indique a todos los hilos que deben terminar limpiamente al vaciar sus colas.
- Cada actor debe correr en su propio `threading.Thread` con `daemon=False`.
- Usar `threading.Lock` para proteger el acceso al contador de estadísticas globales (pedidos completados, tiempo total acumulado).
- Usar `threading.Semaphore` para limitar la cantidad de pedidos que pueden estar en preparación simultáneamente (máximo igual a la cantidad de cocineros activos).
- Cada pedido debe tener un estado representado con una máquina de estados con al menos 4 estados: PENDIENTE → EN_PREPARACION → LISTO → EN_ENTREGA → ENTREGADO.
- Las transiciones de estado deben realizarse de forma atómica usando un `Lock` por pedido.
- El sistema debe correr durante un tiempo total configurable (por ejemplo, 60 segundos de simulación) y luego iniciar el apagado ordenado.
- Al finalizar, imprimir un resumen de métricas:
 - Total de pedidos generados, completados y descartados.
 - Tiempo promedio de preparación.
 - Tiempo promedio de entrega.
 - Tiempo promedio total (desde creación hasta entrega).
 - Throughput: pedidos completados por minuto.
- Todo el logging debe incluir: timestamp, nombre del thread, ID del pedido y estado actual.
- El código debe estar dividido en clases: Pedido, Cliente, Cocinero, Repartidor, MonitorSistema, SistemaDelivery.

RESTRICCIONES:

- No usar `asyncio` como mecanismo principal de concurrencia; el modelo debe basarse en threads reales (`threading`). `asyncio` puede usarse opcionalmente para comparación en una sección aparte claramente delimitada.
- No usar variables globales sin protección de `Lock`. Toda variable compartida entre threads debe estar protegida.
- No usar `time.sleep(0)` como `yield`; los threads deben bloquearse en las colas o en sincronizadores, no en `busy-waiting`.

- No usar `Thread.stop()` ni `Thread.terminate()`; el apagado debe ser cooperativo mediante `shutdown_event`.
- El código no debe tener race conditions detectables: proteger todos los accesos a estado compartido.
- No debe haber posibilidad de deadlock: el orden de adquisición de locks debe ser consistente y documentado en comentarios.
- La cantidad mínima de threads simultáneos en la simulación debe ser: 3 clientes + 4 cocineros + 3 repartidores + 1 monitor = 11 threads mínimo.
- El código debe incluir docstrings en todas las clases y métodos públicos.
- No se permite el uso de la keyword global. Toda variable compartida debe ser atributo de la clase `SistemaDelivery` y pasada por referencia a los actores.

2.3.1. Código generado por Gemini

El modelo generó un script de aproximadamente 250 líneas estructurado en seis clases principales: `EstadoPedido`, `Pedido`, `SistemaDelivery`, `Cliente`, `Cocinero`, `Repartidor` y `MonitorSistema`. Toda la lógica de negocio y estado compartido se encapsuló en la clase `SistemaDelivery`, evitando por completo el uso de variables globales.

Fortalezas: Excelente separación de responsabilidades. Se implementó contrapresión natural bloqueando a los productores cuando la cola alcanza su `maxsize`. El apagado cooperativo se resolvió de forma elegante mediante `threading.Event`, permitiendo que el sistema drene los pedidos activos antes de finalizar. Los locks se adquieren de forma independiente sin anidamiento peligroso.

Debilidades: El sistema adolece de las limitaciones propias del Global Interpreter Lock (GIL) de CPython, aunque al ser una simulación I/O-bound (basada en `time.sleep()`), el impacto en el rendimiento es nulo. Se observa que el método `qsize()` de las colas se utiliza para telemetría; aunque no es estrictamente atómico ni confiable para la lógica de control, su uso en el monitor puramente informativo es aceptable.

2.3.2. Código generado por Sonnet

El modelo generó ~500 líneas de código Python en seis clases: `Pedido` (máquina de estados + Lock propio), `Cliente`, `Cocinero`, `Repartidor` (`threading.Thread`), `MonitorSistema` y `SistemaDelivery` (coordinador). El programa se ejecuta correctamente en Python 3.10 usando `psutil` como única dependencia instalable.

Fortalezas: separación de responsabilidades por clase, bloqueo automático del productor al alcanzar la capacidad máxima de la cola, orden de adquisición de locks documentado en el encabezado del módulo, apagado cooperativo con `threading.Event`.

Debilidades: el GIL de Python limita la paralelización real en tareas CPU-bound (sin impacto aquí por ser I/O-bound); `qsize()` no es atómico respecto a la impresión del monitor; ausencia de pruebas unitarias concurrentes.

2.3.3. Código generado por GPT

El modelo generó ~380 líneas de código Python en seis clases: `Pedido` (máquina de estados con cinco transiciones validadas + Lock propio por instancia), `Cliente` (`threading.Thread` productor), `Cocinero` (`threading.Thread` consumidor con Semaphore de cocina), `Repartidor` (`threading.Thread` consumidor final con actualización de métricas), `MonitorSistema` (`threading.Thread` de observabilidad periódica) y `SistemaDelivery` (orquestador). El programa se ejecuta correctamente en Python 3.10; `psutil` es la única dependencia instalable y se incorpora con fallback silencioso, por lo que su ausencia no interrumpe la simulación.

Fortalezas: Separación de responsabilidades por clase; generación de IDs thread-safe mediante `_id_lock` en el orquestador; `kitchen_semaphore` limita la concurrencia real en cocina independientemente del tamaño de la cola; `cambiar_estado` rechaza transiciones ilegales sin lanzar excepción, preservando la integridad de la máquina de estados; apagado cooperativo implementado con `threading.Event` + `threading.Timer`, sin `daemon=True`; el orden de adquisición de locks (`pedido.lock` → `metrics_lock`) está documentado en el docstring de `Repartidor`.

Debilidades: `Cliente` aplica `put(timeout=1)` y descarta el pedido ante `queue.Full`, lo que introduce pérdida silenciosa de pedidos bajo carga sostenida en lugar de backpressure real; `Cocinero` llama a `cola_listos.put(pedido)` sin timeout, lo que puede bloquear el hilo indefinidamente si la cola de listos está llena y el `shutdown_event` ya fue activado; `qsize()` no es atómico respecto a la lectura del monitor, por lo que los valores imprimibles son estimaciones; `MonitorSistema` recibe una copia de threads en el momento de su construcción (`self.threads + []`), de modo que no se incluye a sí mismo en la lista y cualquier hilo agregado después queda fuera del chequeo de finalización; ausencia de pruebas unitarias

concurrentes; el GIL de Python limita la paralelización real en tareas CPU-bound, aunque no tiene impacto aquí por ser una simulación I/O-bound.

2.3.4. Código generado por Deepseek

El modelo generó ~483 líneas de código Python en siete clases: PedidoLogFormatter (formateador de logging con campos extra garantizados), Pedido (máquina de estados con Lock propio que eleva ValueError en transiciones inválidas), Cliente, Cocinero, Repartidor y MonitorSistema (todos threading.Thread) y SistemaDelivery (orquestador), más una función main() con manejo de KeyboardInterrupt. El programa se ejecuta correctamente en Python 3.10.

Fortalezas: Separación de responsabilidades por clase; configuración centralizada como constantes de módulo; apagado ordenado por capas (clientes → cocineros → repartidores → monitor) con join() secuencial; manejo explícito de KeyboardInterrupt en main() con joins acotados a 2 segundos; task_done() invocado en cocineros y repartidores, dejando las colas compatibles con Queue.join(); estadísticas finales con promedio, mínimo y máximo del tiempo de ciclo; PedidoLogFormatter previene KeyError en el logger cuando los campos extra están ausentes.

Debilidades: psutil se importa sin try/except, por lo que la ausencia del paquete interrumpe el arranque del programa; Cliente y MonitorSistema usan time.sleep() no interrumpible en lugar de shutdown_event.wait(), lo que retrasa la respuesta al apagado hasta el intervalo completo (hasta 3s y 2s respectivamente); Cocinero extrae el pedido de la cola antes de adquirir el semáforo, dejando el pedido en un estado intermedio si el semáforo tarda en otorgarse; Cocinero y Repartidor también usan time.sleep() durante la cocción y el traslado, ignorando el shutdown_event; si cola_listos.put(pedido, timeout=5.0) expira en Cocinero, el pedido queda permanentemente en estado LISTO sin reencole ni compensación; en Repartidor, pedido.timestamp_entrega se lee fuera del lock de pedido para calcular métricas (acceso técnicamente no sincronizado); qsize() no es atómico respecto a la impresión del monitor; ausencia de pruebas unitarias concurrentes; el GIL limita la paralelización real en tareas CPU-bound, sin impacto aquí por ser I/O-bound.

3. Resultados

3.1. Métricas de evaluación

En este trabajo se ha elegido medir dos dimensiones de la calidad de los códigos generados: Complejidad ciclomática (CC) y mantenibilidad (MI). La CC mide el número de caminos lógicamente independientes (ramificaciones). A mayor CC, más casos de prueba y más riesgo de defectos. El Índice de Mantenibilidad (MI) combina CC, Halstead y tamaño para ofrecer un indicador agregado de facilidad de mantenimiento. Estas métricas ayudan a detectar “puntos calientes” (funciones que conviene refactorizar antes).

3.2.1. Resultados obtenidos en código generado por Gemini

El código presentó una Complejidad Ciclomática (CC) promedio de ~2.10 (según los fundamentos de McCabe, 1976), caracterizada por el uso de bucles planos y la ausencia de anidamiento profundo. Su Índice de Mantenibilidad (MI) rondó los ~55.00 sobre 100 (Oman & Hagemeister, 1992), lo que indica una alta legibilidad favorecida por el uso de docstrings y un correcto encapsulamiento. Durante la simulación, se registraron 12 threads simultáneos ($\$3C + 4Co + 3R + 1M + \text{main}\$$). A nivel de rendimiento, el uso de CPU promedio fue inferior al 0.5%, un comportamiento predecible al ser un sistema I/O-bound, mientras que el consumo de memoria RSS se mantuvo entre 15 y 20 MB, reflejando el uso estándar del intérprete de Python sin dependencias pesadas. Finalmente, se estimó un total de ~2800 tokens (aproximadamente 800 correspondientes al prompt y 2000 al código generado) y se requirieron 0 refinamientos, logrando que el código cumpliera con los requisitos funcionales en el primer intento (zero-shot).

3.2.2. Resultados obtenidos en código generado por Sonnet

El código analizado presentó una Complejidad Ciclomática (CC) promedio de ~2.42, calculada mediante radon cc según los fundamentos de McCabe (1976). Por su parte, el Índice de Mantenibilidad (MI) se situó en ~39.82 sobre 100, obtenido a través de radon mi siguiendo el modelo de Oman & Hagemeister (1992). Durante su ejecución, la función threading.active_count() registró un total de 12 threads simultáneos (correspondientes a $\$3C + 4Co + 3R + 1M + \text{main}\$$). A nivel de rendimiento del sistema, el uso de CPU promedio en la simulación fue mínimo (~0.2%), medido con psutil.cpu_percent(), y el consumo de

memoria RSS se mantuvo ligero, oscilando entre ~12 y 15 MB según `psutil.memory_info().rss`. En relación con el proceso de generación mediante IA, se estimó un uso total de tokens de ~1515 para el prompt y 5903 para el código (verificado con un contador online), y se requirió 1 refinamiento específico para integrar las herramientas de medición de CPU y memoria. Finalmente, el análisis profundo de la concurrencia determinó que no existen deadlocks (basado en el análisis de lock ordering) ni race conditions (gracias a la documentación e implementación de locks por pedido y `stats_lock`). No obstante, se advierte que el starvation es posible (de forma leve) al emplear una cola sin prioridades, una debilidad que resulta mitigable si se opta por utilizar `PriorityQueue`.

3.2.3. Resultados obtenidos en código generado por GPT

El código presentó una Complejidad Ciclomática (CC) promedio de ~3.9 (según los fundamentos de McCabe, 1976), ya que la mayoría de los métodos son bucles con una o dos bifurcaciones. Su Índice de Mantenibilidad (MI) rondó los ~46,86 sobre 100 (Oman & Hagemester, 1992), lo que indica una legibilidad media, debido a la falta de algunos docstrings. Durante la simulación, se registraron 12 threads simultáneos ($3C + 4Co + 3R + 1M + \text{main}$). A nivel de rendimiento, el uso de CPU promedio fue inferior al 10.61%, un comportamiento predecible al ser un sistema I/O-bound, mientras que el consumo de memoria RSS se mantuvo entre 15 y 20 MB, reflejando el uso estándar del intérprete de Python sin dependencias pesadas. Finalmente, se estimó un total de ~5955 tokens (aproximadamente 2547 correspondientes al prompt y 3408 al código generado) y se requirieron 0 refinamientos, logrando que el código cumpliera con los requisitos funcionales en el primer intento (zero-shot).

3.2.4. Resultados obtenidos en código generado por Deepseek

El código presentó una Complejidad Ciclomática (CC) promedio de ~3.5 (según los fundamentos de McCabe, 1976), ya que distribuye más lógica en funciones auxiliares cortas. Su Índice de Mantenibilidad (MI) rondó los ~31,82 sobre 100 (Oman & Hagemester, 1992), debido a mucha densidad de strings de logging con f-strings largos en prácticamente cada paso de cada actor, constantes de configuración globales que aumentan el volumen Halstead, y la clase `PedidoLogFormatter` que agrega LOC sin lógica

sustancial. Durante la simulación, se registraron 12 threads simultáneos ($3C + 4Co + 3R + 1M + \text{main}$). A nivel de rendimiento, el uso de CPU promedio fue inferior al 12.43%, un comportamiento predecible al ser un sistema I/O-bound, mientras que el consumo de memoria RSS se mantuvo entre 15 y 20 MB, reflejando el uso estándar del intérprete de Python sin dependencias pesadas. Finalmente, se estimó un total de ~6670 tokens (aproximadamente 2523 correspondientes al prompt y 4147 al código generado) y se requirieron 0 refinamientos, logrando que el código cumpliera con los requisitos funcionales en el primer intento (zero-shot).

4. Conclusiones

Las herramientas de IA deben ser utilizadas con buen juicio y criterio a la hora de producir código propio o realizar ajustes en código ajeno, ya que esto garantizará un proyecto más robusto y mantenible, que lógicamente devendrá en una mayor calidad del producto software. Además de que un buen análisis y especificación del sistema a desarrollar provocará que la generación de código sea más eficaz funcionalmente y eficiente en cuanto al uso de tokens.

Referencias

- [1] ACM, "Concurrent computing," ACM Digital Library, 2024.
- [2] G. R. Andrews, Foundations of Multithreaded, Parallel, and Distributed Programming, Addison-Wesley, 2000.
- [3] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.
- [4] E. W. Dijkstra, "Cooperating sequential processes," en Programming Languages, Academic Press, 1968, pp. 43–112.
- [5] T. J. McCabe, "A complexity measure," IEEE Trans. Software Eng., vol. SE-2, no. 4, pp. 308–320, 1976.
- [6] P. Oman y J. Hagemester, "Metrics for assessing maintainability," en Proc. ICSM, IEEE, 1992, pp. 337–344.
- [7] M. H. Halstead, Elements of Software Science, Elsevier, 1977.
- [8] D. Beazley, "Understanding the Python GIL," en Proc. PyCon, Chicago, 2010.
- [9] G. J. Holzmann, The SPIN Model Checker, Addison-Wesley, 2003.

[10] ClaudeRules, "Claude Tokenizer — Free Token Counter for Claude Code," clauderules.net, [en línea]. Fecha de consulta: 9 de mayo de 2026. Disponible: <https://clauderules.net/tools/token-counter>

